Faculty of Computer Applications & IT
Class Test -I

Subject: OOP Using Python **Marks:50**

Course: M. Sc.(IT)- Sem-I **Time: 1 hour**

Roll No: _____ **Machine ID:**

Set-1

Develop a Student Result Management System in Python using strings, lists, nested lists, loops, and conditional statements.

The program should:

1. Store the name of at least 5 students along with their marks in 3 subjects using a nested list.
2. Calculate the total marks and percentage of each student.
3. Assign a grade based on the percentage.
4. Display the result of every student.
5. Find and display the student with the highest total marks.
6. Store the names of students who have failed in a separate list.
7. Display the overall class average.

Faculty of Computer Applications & IT
Class Test -I

Subject: OOP Using Python **Marks:50**

Course: M. Sc.(IT)- Sem-I **Time: 1 hour**

Roll No: _____ **Machine ID:**

Set- 2

Develop a **Shopping Cart and Billing System** in Python using strings, lists, nested lists, loops, and conditional statements.

The program should:

1. Store at least 5 products with product name, price, and quantity using a nested list.
2. Display all available products.
3. Calculate the amount for each product based on price and quantity.
4. Calculate the total bill.
5. Apply a discount based on the total purchase amount.
6. Store the names of products purchased in a separate list.
7. Display the final bill with discount and net payable amount.

Faculty of Computer Applications & IT
Class Test -I

Subject: OOP Using Python **Marks:50**

Course: M. Sc.(IT)- Sem-I **Time: 1 hour**

Roll No: _____ **Machine ID:**

SET-3

Develop a Library Management System in Python using strings, lists, nested lists, loops, and conditional statements.

The program should:

1. Store at least 5 books with book title, author name, and availability status using a nested list.
2. Display all available books.
3. Allow the user to search for a book by title.
4. Display whether the requested book is available or issued.
5. Allow the user to issue an available book.
6. Store issued book titles in a separate list.
7. Display the final list of available and issued books.

Faculty of Computer Applications & IT
Class Test -I

Subject: OOP Using Python **Marks:50**

Course: M. Sc.(IT)- Sem-I **Time: 1 hour**

Roll No: _____ **Machine ID:**

SET -04

Develop an **Employee Salary Management System** in Python using strings, lists, nested lists, loops, and conditional statements.

The program should:

1. Store at least 5 employees with employee name, department, basic salary, and allowance using a nested list.
2. Calculate the gross salary of each employee.
3. Categorize employees as Low, Medium, or High salary based on gross salary.
4. Display the salary details of all employees.
5. Find the employee receiving the highest salary.
6. Store the names of high-salary employees in a separate list.
7. Calculate and display the average salary of all employees.

Faculty of Computer Applications & IT
Class Test -I

Subject: OOP Using Python **Marks:50**

Course: M. Sc.(IT)- Sem-I **Time: 1 hour**

Roll No: _____ **Machine ID:**

SET-05

Develop a **Movie Ticket Booking System** in Python using strings, lists, nested lists, loops, and conditional statements.

The program should:

1. Store at least 5 movies with movie name, ticket price, and available seats using a nested list.
2. Display the available movies.
3. Allow the user to select a movie and enter the number of tickets required.
4. Check whether sufficient seats are available.
5. Calculate the total ticket cost.
6. Store the names of booked movies in a separate list.
7. Display the booking details and total amount.

Faculty of Computer Applications & IT
Class Test -I

Subject: OOP Using Python **Marks:50**

Course: M. Sc.(IT)- Sem-I **Time: 1 hour**

Roll No: _____ **Machine ID:**

SET- 06

Develop an Online Food Ordering System in Python using strings, lists, nested lists, loops, and conditional statements.

The program should:

1. Store at least 6 food items with item name, category, and price using a nested list.
2. Display the available food items.
3. Allow the user to select food items and specify quantities.
4. Calculate the cost of each selected item.
5. Calculate the total order amount.
6. Apply a discount if the order amount crosses a specified limit.
7. Store the selected food items in a separate list and display the final order summary.